



AI VIET NAM

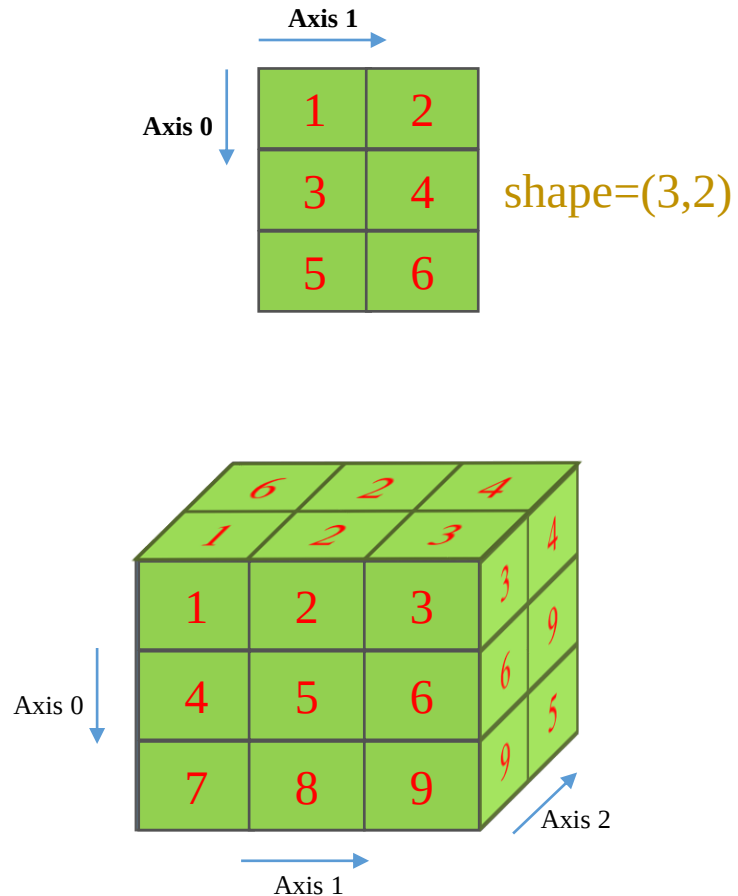
@aivietnam.edu.vn

Introduction to Numpy

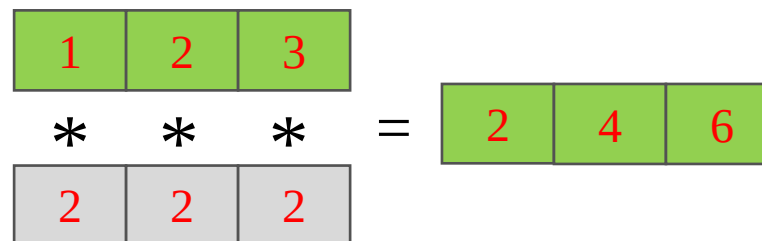
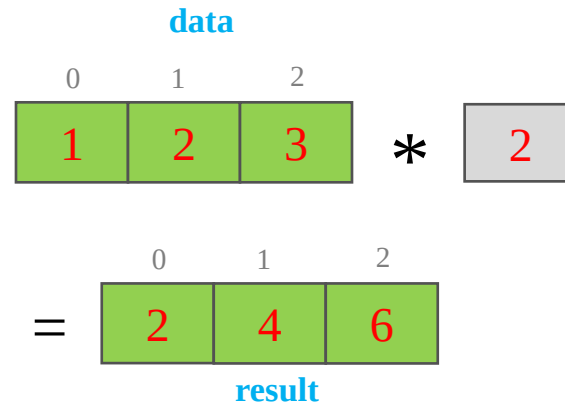
Quang-Vinh Dinh
PhD in Computer Science

Objectives

Indexing



Broadcasting



Examples

Feature	Label
Petal_Length	Category
1.4	0
1	0
1.5	0
3	1
3.8	1
4.1	1

One-hot encoding for label

$$y = 0 \rightarrow \mathbf{y} = [1 \ 0]$$

$$y = 1 \rightarrow \mathbf{y} = [0 \ 1]$$



Outline

SECTION 1

Introduction

SECTION 2

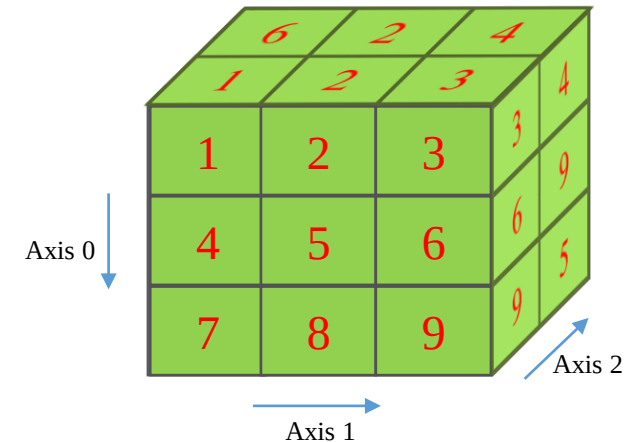
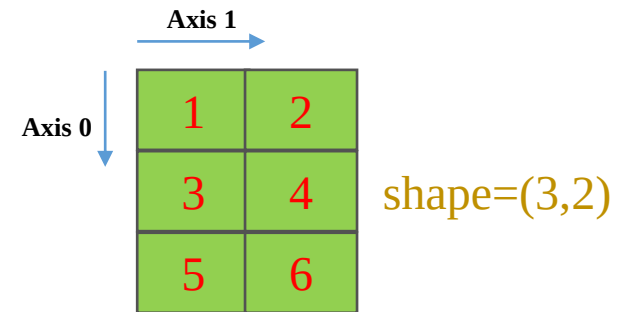
Common Functions

SECTION 3

Indexing and Broadcasting

SECTION 4

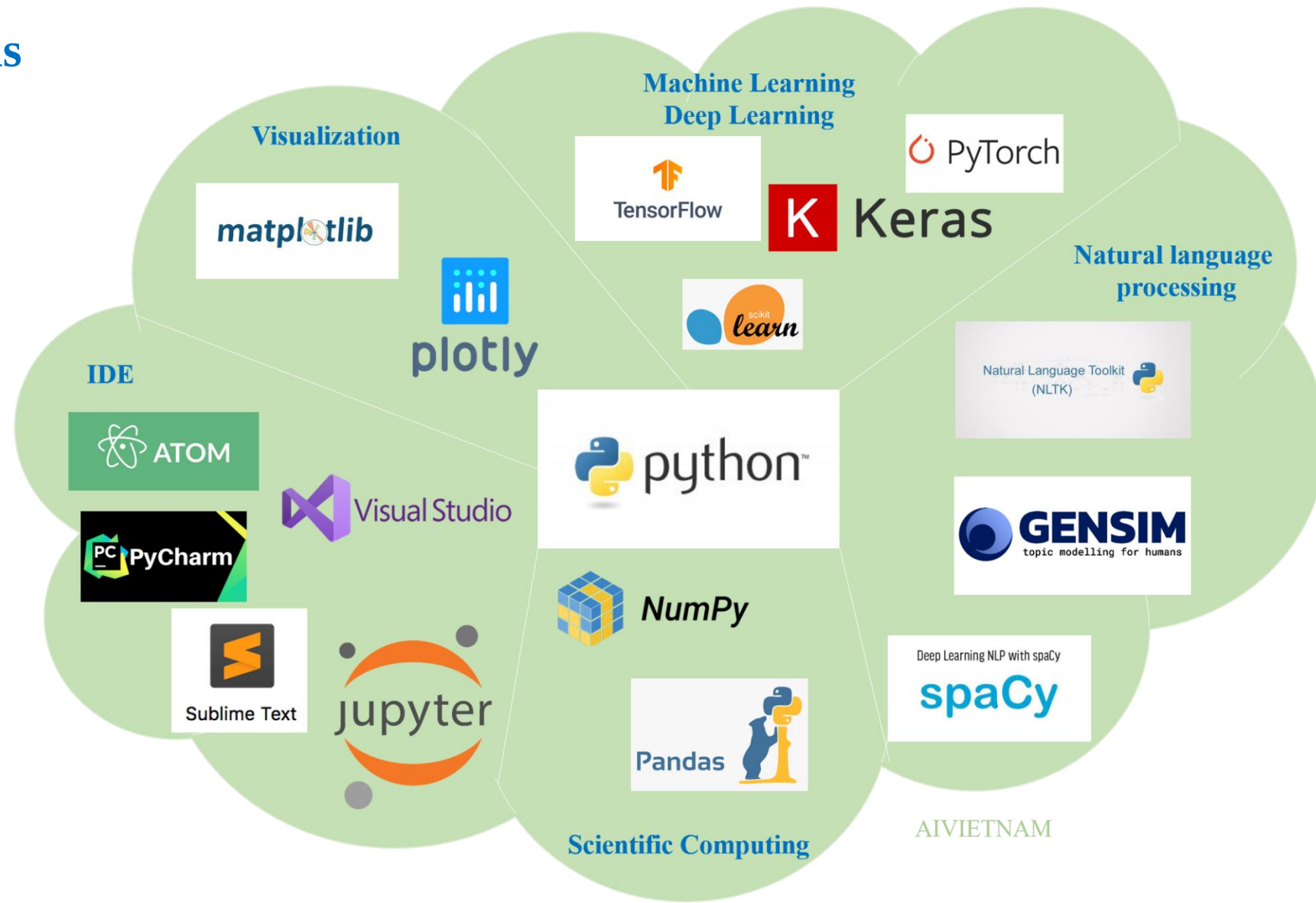
Examples



- ❖ Numpy is a Python library
- ❖ For scientific computations

NumPy (pronounced [/ˈnʌmpaɪ/](#) *NUM-py*) is a [library](#) for the [Python](#) programming language

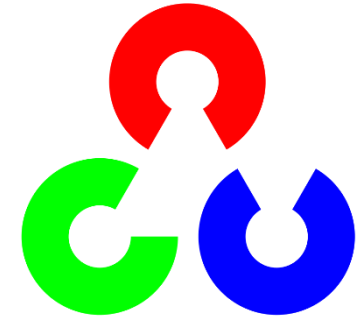
Original author(s)	Travis Oliphant
Developer(s)	Community project
Initial release	As Numeric, 1995; as NumPy, 2006
Stable release	2.0.0 ^[1] / 16 June 2024; 12 days ago
Repository	github.com/numpy/numpy ↗ ✎
Written in	Python, C
Operating system	Cross-platform
Type	Numerical analysis
License	BSD ^[2]
Website	numpy.org ↗





❖ Data type for common libraries

 PyTorch

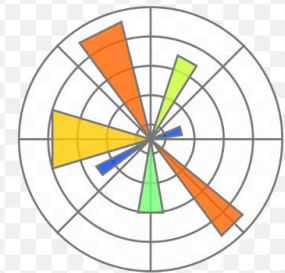

OpenCV



NumPy



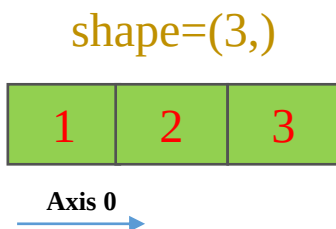
TensorFlow



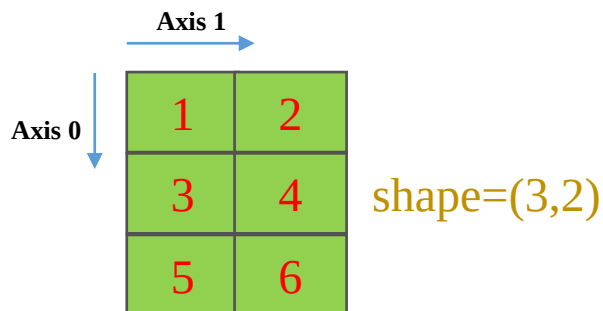
matplotlib

Introduction

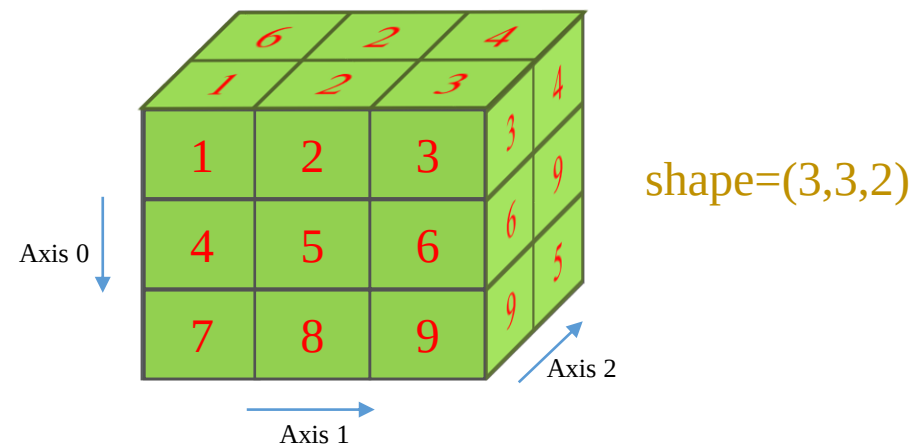
❖ Numpy arrays are multi-dimensional arrays



1D array



2D array



3D array

```
4 import numpy as np
5
6 # tạo list
7 list1D = [1,2,3]
8
9 # tạo ndarray
10 data = np.array(list1D)
11
12 print(data)
13 print(data.shape)
```

```
[1 2 3]
(3,)
```

```
4 import numpy as np
5
6 # tạo list
7 list2D = [[1,2],[3,4],[5,6]]
8
9 # tạo ndarray
10 data = np.array(list2D)
11
12 print(data)
13 print(data.shape)
```

```
[[1 2]
 [3 4]
 [5 6]]
(3, 2)
```

```
4 import numpy as np
5
6 # tạo list
7 list3D = [[[1,6], [2,2], [3,4]],
8            [[4,7], [5,2], [6,9]],
9            [[7,7], [8,2], [9,5]]]
10
11 # tạo ndarray
12 data = np.array(list3D)
13
14 #print(data)
15 print(data.shape)
```

```
(3, 3, 2)
```

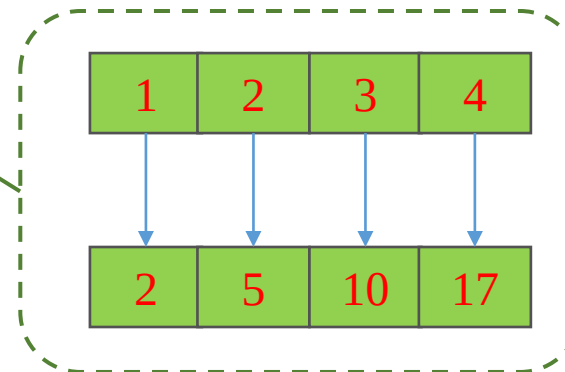
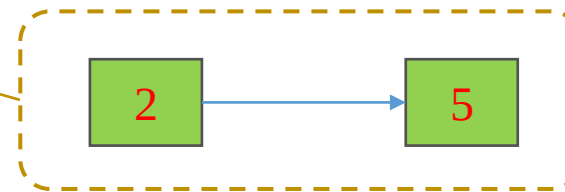
❖ Why do I need Numpy?

```
def f_function(number):  
    result = number**2 + 1  
    return result  
  
def f_function_list(numbers):  
    result = []  
    for n in numbers:  
        result.append(f_function(n))  
    return result
```

```
# test  
numbers = [1, 2, 3, 4]  
print(f_function_list(numbers))
```

✓ 0.0s

[2, 5, 10, 17]



❖ Why do I need Numpy?

```
def f_function(number):  
    result = number**2 + 1  
    return result  
  
def f_function_list(numbers):  
    result = []  
    for n in numbers:  
        result.append(f_function(n))  
    return result  
  
# test  
numbers = [1, 2, 3, 4]  
print(f_function_list(numbers))
```

✓ 0.0s

[2, 5, 10, 17]

```
import numpy as np
```

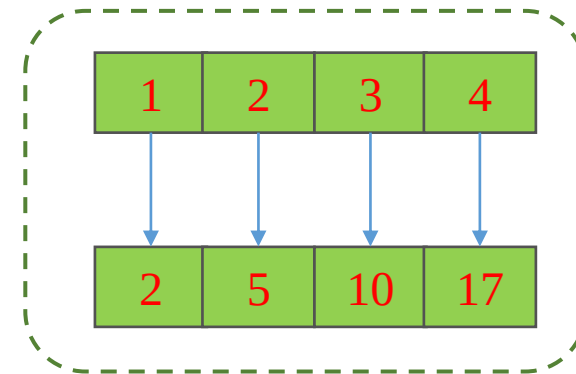
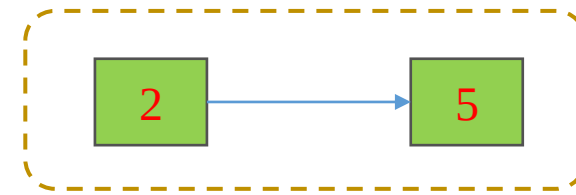
```
def f_function(number):  
    result = number**2 + 1  
    return result
```

```
factorial_np = np.vectorize(f_function)
```

```
# test  
numbers = [1, 2, 3, 4]  
numbers_array = np.array(numbers)  
print(factorial_np(numbers_array))
```

✓ 0.0s

[2 5 10 17]



❖ Why do I need Numpy?

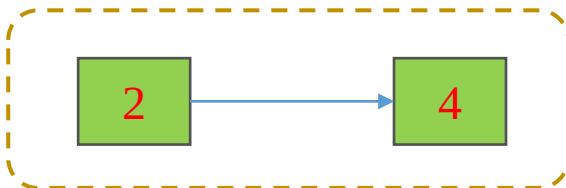
```
def f_function(number):
    result = number**2 + 1
    return result

def f_function_list(numbers):
    result = []
    for n in numbers:
        result.append(f_function(n))
    return result

# test
numbers = [1, 2, 3, 4]
print(f_function_list(numbers))
```

✓ 0.0s

[2, 5, 10, 17]



```
import numpy as np

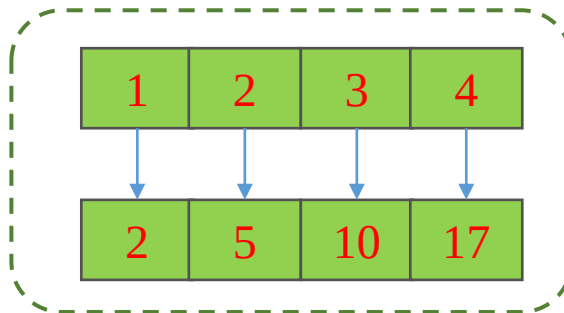
def f_function(number):
    result = number**2 + 1
    return result

factorial_np = np.vectorize(f_function)

# test
numbers = [1, 2, 3, 4]
numbers_array = np.array(numbers)
print(factorial_np(numbers_array))
```

✓ 0.0s

[2 5 10 17]



```
import numpy as np

# data
numbers = [1, 2, 3, 4]
numbers_np = np.array(numbers)

# compute
numbers_np = numbers_np**2 + 1
print(numbers_np)
```

✓ 0.0s

[2 5 10 17]

Implementation view
implement for each element

Execution view
run for all the elements

Outline

SECTION 1

Introduction

SECTION 2

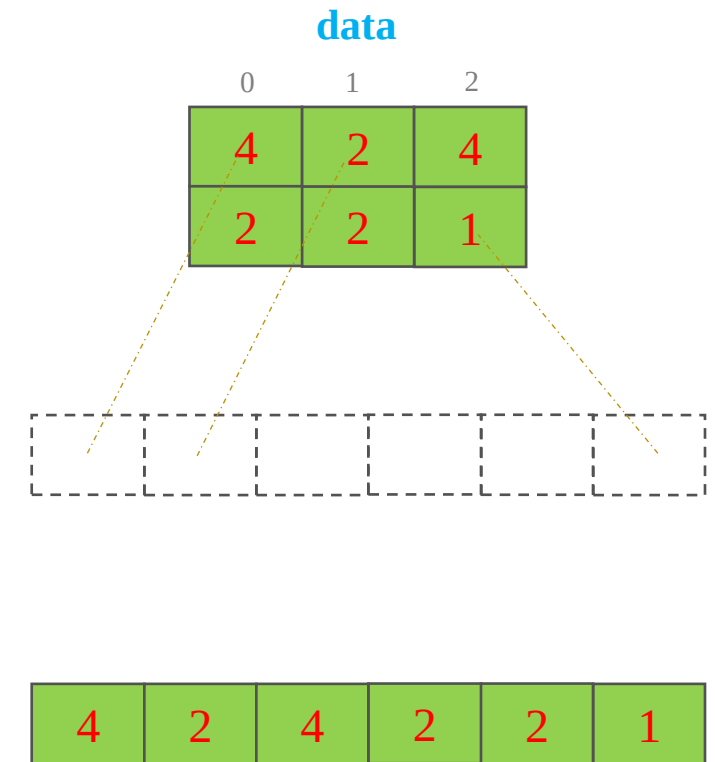
Common Functions

SECTION 3

Indexing and Broadcasting

SECTION 4

Examples



❖ Create Numpy arrays

zeros() function

	0	1	2
0	0	0	0
1	0	0	0

```
1 # aivietnam.ai
2 # Tạo một numpy array
3 # với tất cả phần tử là 0
4
5 import numpy as np
6
7 # shape: 2 dòng, 3 cột
8 arr = np.zeros((2,3))
9 print(arr)
```

```
[[0. 0. 0.]
 [0. 0. 0.]]
```

ones() function

	0	1	2
0	1	1	1
1	1	1	1

```
1 # aivietnam.ai
2 # Tạo một numpy array với
3 # tất cả phần tử là 1
4
5 import numpy as np
6
7 # numpy.ones(shape)
8 # shape: 2 dòng, 3 cột
9 arr = np.ones((2,3))
10 print(arr)
```

```
[[1. 1. 1.]
 [1. 1. 1.]]
```

full() function

	0	1	2
0	9	9	9
1	9	9	9

```
1 # aivietnam.ai
2 # Tạo một numpy array với tất
3 # cả phần tử là hằng số fill_value
4
5 import numpy as np
6
7 # numpy.full(shape, fill_value)
8 # shape: 2 dòng, 3 cột
9 arr = np.full((2,3), 9)
10 print(arr)
```

```
[[9 9 9]
 [9 9 9]]
```

❖ Create Numpy arrays

eye() function

	0	1	2
0	1	0	0
1	0	1	0
2	0	0	1

```
1 # aivietnam.ai
2 # Tạo một numpy array với đường chéo là số 1
3 # số 0 được điền vào những ô phần tử còn lại
4
5 import numpy as np
6
7 # numpy.eye(N)
8 # shape: 3 dòng, 3 cột
9 arr = np.eye(3)
10 print(arr)
```

```
[[1.  0.  0.]
 [0.  1.  0.]
 [0.  0.  1.]]
```

random() function

	0	1	2
0	0.574	0.682	0.704
1	0.806	0.844	0.799

```
1 # aivietnam.ai
2 # Tạo một numpy array với
3 # giá trị ngẫu nhiên
4
5 import numpy as np
6
7 # np.random.random(size)
8 # shape: 2 dòng, 3 cột; với
9 # phần tử có giá trị ngẫu nhiên
10 arr = np.random.random((2,3))
11 print(arr)
```

```
[[0.57488062 0.68266312 0.70438569]
 [0.80661973 0.84413356 0.79905247]]
```

arange(start, end, step)

arange() function

	0	1	2	3	4
arr1 =	0	1	2	3	4
	0	1	2		
arr2 =	0	2	4		

```
1 # aivietnam.ai
2 import numpy as np
3
4 # np.arange(start=0, stop, step=1)
5 arr1 = np.arange(5)
6 print(arr1)
7
8 arr2 = np.arange(0, 5, 2)
9 print(arr2)
```

```
[0 1 2 3 4]
[0 2 4]
```

❖ reshape function

reshape() function

data

1	2	3
4	5	6

data_rs

1	2
3	4
5	6

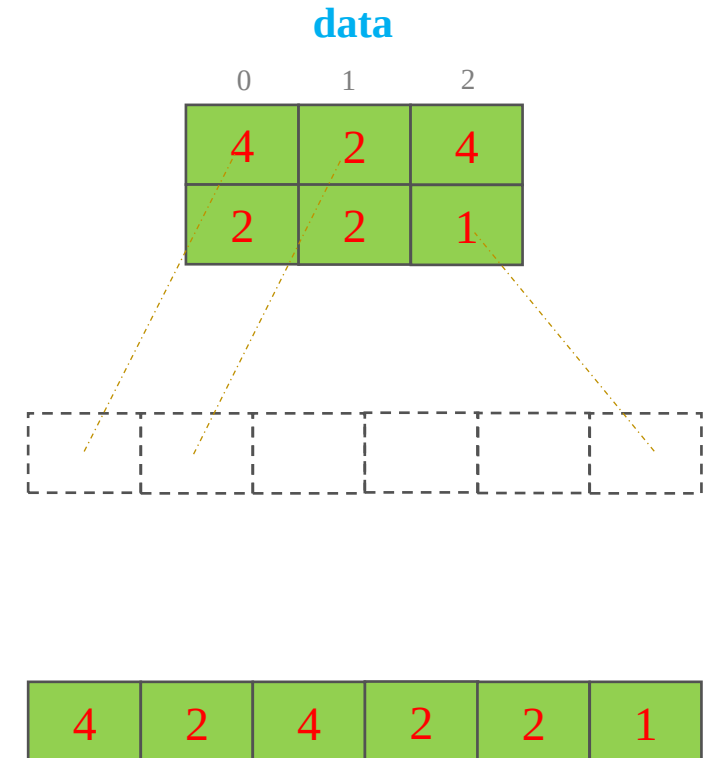
```
1 # aivietnam.ai
2 import numpy as np
3
4 # tạo list
5 l = [[1,2,3],
6       [4,5,6]]
7
8 # tạo ndarray
9 data = np.array(l)
10 print('data\n', data)
11 print('data shape\n', data.shape)
12
13 # reshape
14 data_rs = np.reshape(data, (3,2))
15 print('data_rs\n', data_rs)
16 print('data_rs shape\n', data_rs.shape)
```

```
data
[[1 2 3]
 [4 5 6]]
data shape
(2, 3)
data_rs
[[1 2]
 [3 4]
 [5 6]]
data_rs shape
(3, 2)
```

❖ reshape and flatten functions

```
1 import numpy as np
2
3 # generate random integers
4 data = np.random.randint(1, 5, (2, 3))
5 print('data: \n', data)
6
7 vector1 = data.flatten()
8 print('vector1: \n', vector1)
9
10 vector2 = data.reshape(-1)
11 print('vector2: \n', vector2)
```

```
data:
[[4 2 4]
 [2 2 1]]
vector1:
[4 2 4 2 2 1]
vector2:
[4 2 4 2 2 1]
```





❖ reshape and flatten functions

```
1 import numpy as np
2
3 # generate random integers
4 data = np.random.randint(1, 9, (2, 2))
5 print('data: \n', data)
```

data:

```
[[4 3]
 [7 1]]
```

```
1 # repeat
2 data_repeat = np.repeat(data, 2, axis=0)
3 print(data_repeat)
```

```
[[4 3]
 [4 3]
 [7 1]
 [7 1]]
```

```
1 # repeat
2 data_repeat = np.repeat(data, 2, axis=1)
3 print(data_repeat)
```

```
[[4 4 3 3]
 [7 7 1 1]]
```

`repeat(data, repeats, axis)`

data

4	3
7	1

repeat_axis_0

4	3
4	3
7	1
7	1

repeat_axis_1

4	4	3	3
7	7	1	1

Outline

SECTION 1

Introduction

SECTION 2

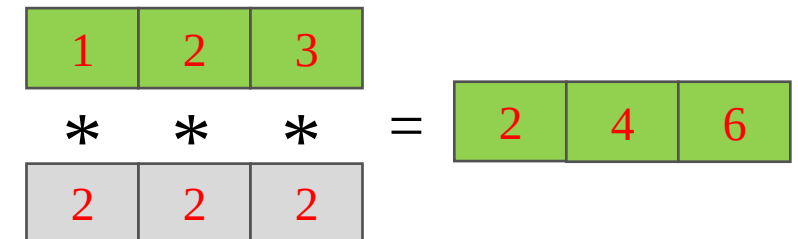
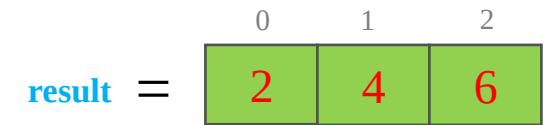
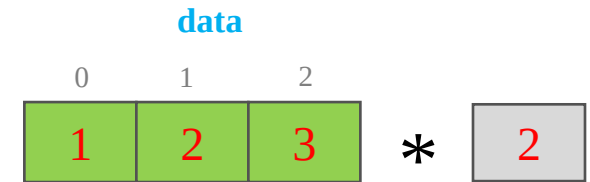
Common Functions

SECTION 3

Indexing and Broadcasting

SECTION 4

Examples



❖ Slicing

`arr[for_axis_0, for_axis_1, ...]`

`'.'`: get all the elements

`'a:b'`: get the elements from a^{th} to $(b^{th}-1)$

data

	0	1
0	1	2
1	3	4
2	5	6

data[0,1]

	0	1
0	1	2
1	3	4
2	5	6

data[1:3]

	0	1
0	1	2
1	3	4
2	5	6

data[0:1,0]

	0	1
0	1	2
1	3	4
2	5	6

data[:,:]

	0	1
0	1	2
1	3	4
2	5	6

```

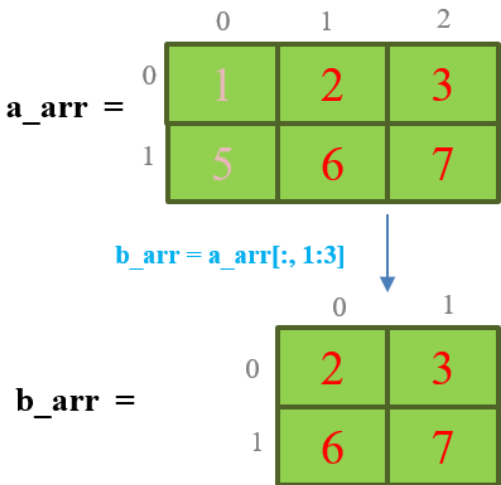
1  # aivietnam.ai
2  import numpy as np
3
4  # Khởi tạo numpy array a_arr
5  a_arr = np.array([[1,2,3],
6                   [5,6,7]])
7
8  # Sử dụng slicing để tạo mảng b_arr
9  # bằng cách lấy tất cả các dòng và cột 1,2
10 b_arr = a_arr[:, 1:3]
11
12 print(a_arr)
13 print(b_arr)

```

```

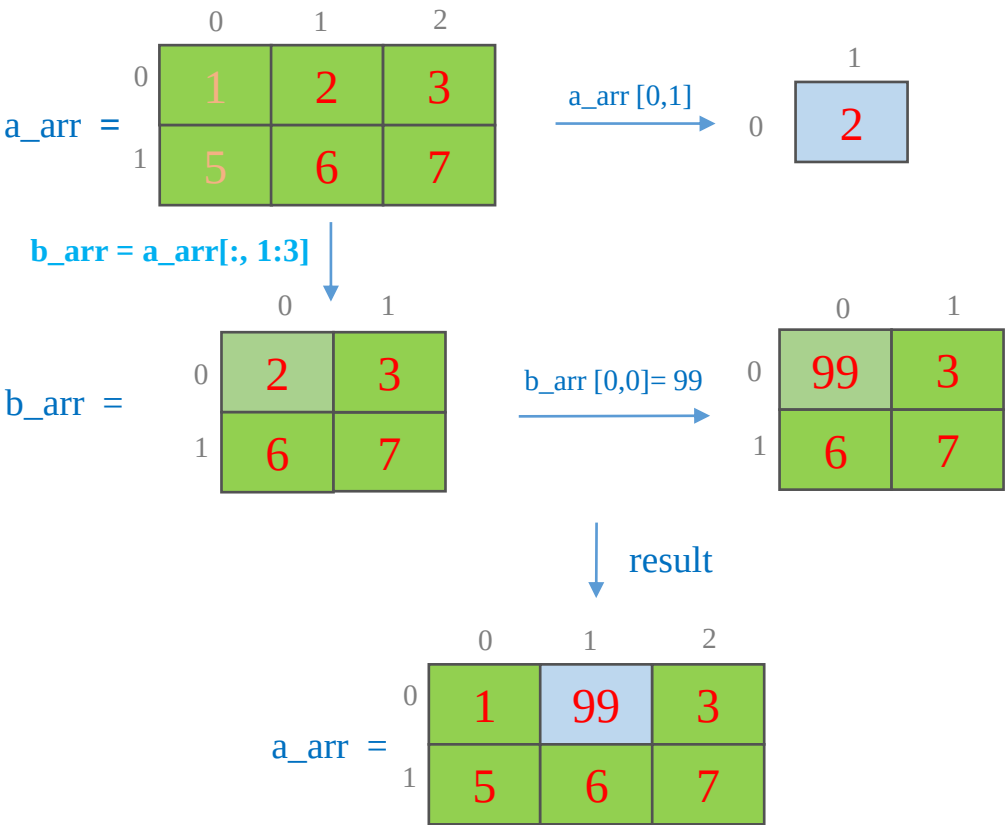
[[1 2 3]
 [5 6 7]]
[[2 3]
 [6 7]]

```



❖ Slicing

❖ Mutable



```
1 # aivietnam.ai
2 import numpy as np
3
4 # Khởi tạo numpy array a_arr
5 a_arr = np.array([[1,2,3],
6                   [5,6,7]])
7 print('a_arr \n', a_arr)
8
9 # Sử dụng slicing để tạo mảng b_arr
10 b_arr = a_arr[:, 1:3]
11 print('b_arr \n', b_arr)
12
13 print('before changing \n', a_arr[0, 1])
14 b_arr[0, 0] = 99
15 print('after changing \n', a_arr[0, 1])
```

a_arr

```
[[1 2 3]
 [5 6 7]]
```

b_arr

```
[[2 3]
 [6 7]]
```

before changing

2

after changing

99



❖ Get a row

		0	1	2	
	0	1	2	3	
	1	5	6	7	
	2	9	10	11	

arr =

		0	1	2	
	0	5	6	7	

row_m1 =

		0	1	2	
	0	5	6	7	

row_m2 =

		0	1	2	
	0	5	6	7	

shape(3,)

shape(1, 3)

```
1 # aivietnam.ai
2 import numpy as np
3
4 # Tạo một numpy array có shape (3, 3) với giá trị
5 # [[ 1  2  3]
6 #  [ 5  6  7]
7 #  [ 9 10 11]]
8 arr = np.array([[1, 2, 3],
9                 [5, 6, 7],
10                [9,10,11]])
11
12 # Hai cách truy cập dữ liệu ở dòng index=1
13 # cách 1: số chiều giảm
14 row_m1 = arr[1, :]
15
16 # cách 2: số chiều được giữ nguyên
17 row_m2 = arr[1:2, :]
18
19 print(row_m1, row_m1.shape)
20 print(row_m1, row_m2.shape)
```

```
[5 6 7] (3,)
[5 6 7] (1, 3)
```

❖ Get a column

		0	1	2	
	0	1	2	3	
	1	5	6	7	
	2	9	10	11	

arr =

		0	1	2	
	0	2	6	10	

col_m1 = (3,)

		0			
	0	2			
	1	6			
	2	10			

col_m2 = (3, 1)

```
1 # aivietnam.ai
2 import numpy as np
3
4 # Tạo một numpy array có shape (3, 3) với giá trị
5 arr = np.array([[1,2,3],
6                 [5,6,7],
7                 [9,10,11]])
8
9 # Hai cách truy cập dữ liệu ở cột index=1 của mảng
10 # cách 1: số chiều giảm
11 col_m1 = arr[:, 1]
12
13 # cách 2: số chiều được giữ nguyên
14 col_m2 = arr[:, 1:2]
15
16 print(col_m1, col_m1.shape)
17 print(col_m2, col_m2.shape)
```

```
[ 2  6 10] (3,)
[[ 2]
 [ 6]
 [10]] (3, 1)
```

❖ Using Lists as indices

		0	1
	0	1	2
arr =	1	3	4
	2	5	6

arr[[0, 1, 2], [0, 1, 0]] =	1	4	5
-----------------------------	---	---	---

arr[[0, 0], [1, 1]] =	2	2
-----------------------	---	---

```

1 # aivietnam.ai
2 import numpy as np
3
4 # tạo arr
5 arr = np.array([[1, 2],
6                 [3, 4],
7                 [5, 6]])
8
9 # lấy giá trị vị trí (0,0), (1,1) và (2,0)
10 out1 = arr[[0, 1, 2], [0, 1, 0]]
11 print('out1:\n', out1)
12
13 # Có thể truy xuất tới 1 phần tử nhiều hơn 1 lần
14 out2 = arr[[0, 0], [1, 1]]
15 print('out2:\n', out2)

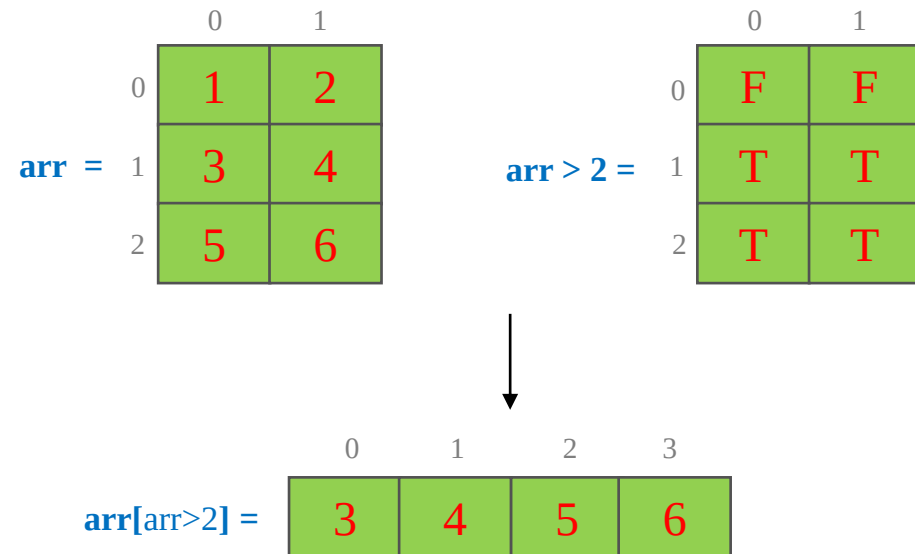
```

```

out1:
[1 4 5]
out2:
[2 2]

```

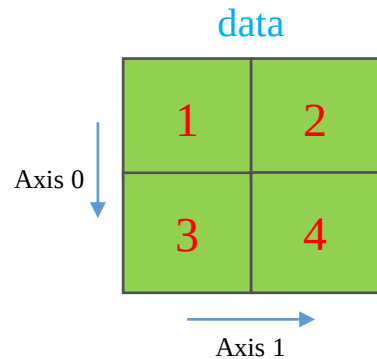
❖ Boolean indices



```
1 # aivietnam.ai
2 import numpy as np
3
4 arr = np.array([[1, 2],
5                 [3, 4],
6                 [5, 6]])
7
8 # tìm các phần tử lớn hơn 2
9 bool_idx = (arr > 2)
10
11 # get satisfying elements
12 result = arr[bool_idx]
13 print(result)
```

[3 4 5 6]

❖ Summation



sum(data, axis=0)



sum(data)



sum(data, axis=1)



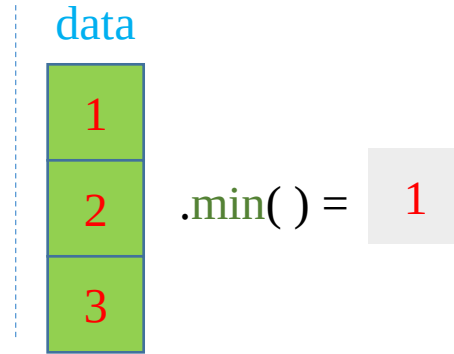
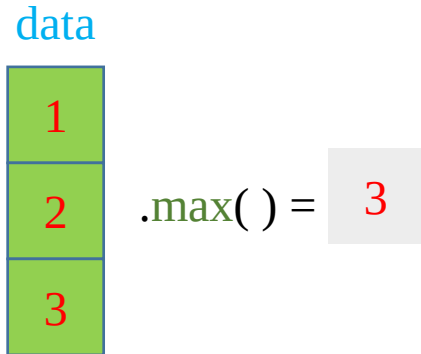
```
1 # aivietnam.ai
2 import numpy as np
3
4 data = np.array([[1,2],
5                  [3,4]])
6
7 # Tổng các phần tử của mảng
8 print(np.sum(data))
9
10 # Tính tổng theo từng cột
11 print(np.sum(data, axis=0))
12
13 # Tính tổng theo từng dòng
14 print(np.sum(data, axis=1))
```

10

[4 6]

[3 7]

❖ Max and min

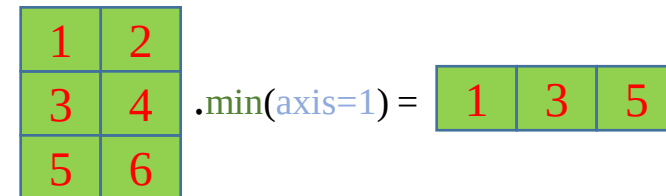
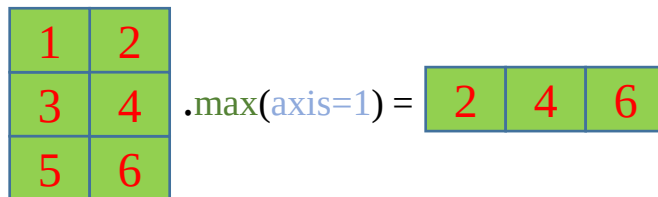
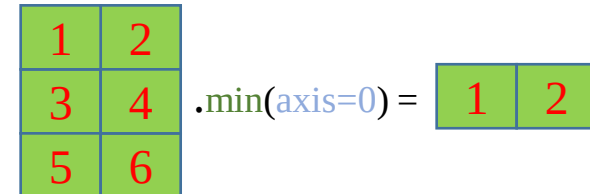
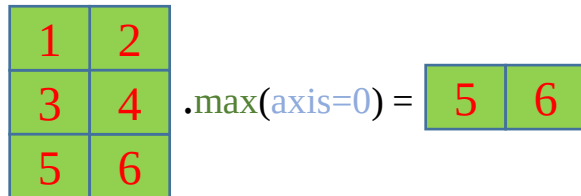


```

1 # aivietnam.ai
2 import numpy as np
3
4 data = np.array([1, 2, 3])
5
6 print(data.max())
7 print(data.min())

```

3
1



❖ Vector and a scalar

data

0	1	2
1	2	3

 *

2

 =

result

0	1	2
2	4	6

data

0	1	2
1	2	3

 -

2

 =

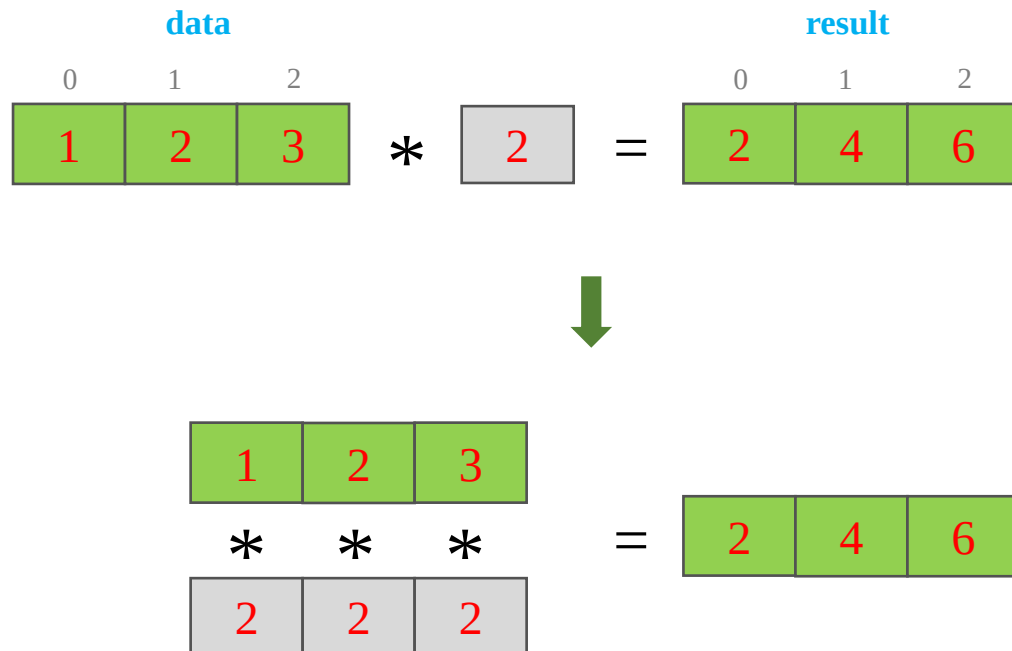
result

0	1	2
-1	0	1

```
1 # aivietnam.ai
2 import numpy as np
3
4 # create data
5 data = np.array([1,2,3])
6 factor = 2
7
8 # broadcasting
9 result_multiplication = data*factor
10 result_minus = data - factor
11
12 print(data)
13 print(result_multiplication)
14 print(result_minus)
```

```
[1 2 3]
[2 4 6]
[-1  0  1]
```

❖ Vector and a scalar



```
1 # aivietnam.ai
2 import numpy as np
3
4 # create data
5 data = np.array([1,2,3])
6 factor = 2
7
8 # broadcasting
9 result_multiplication = data*factor
10 result_minus = data - factor
11
12 print(data)
13 print(result_multiplication)
14 print(result_minus)
```

```
[1 2 3]
[2 4 6]
[-1  0  1]
```

❖ Matrix and vector

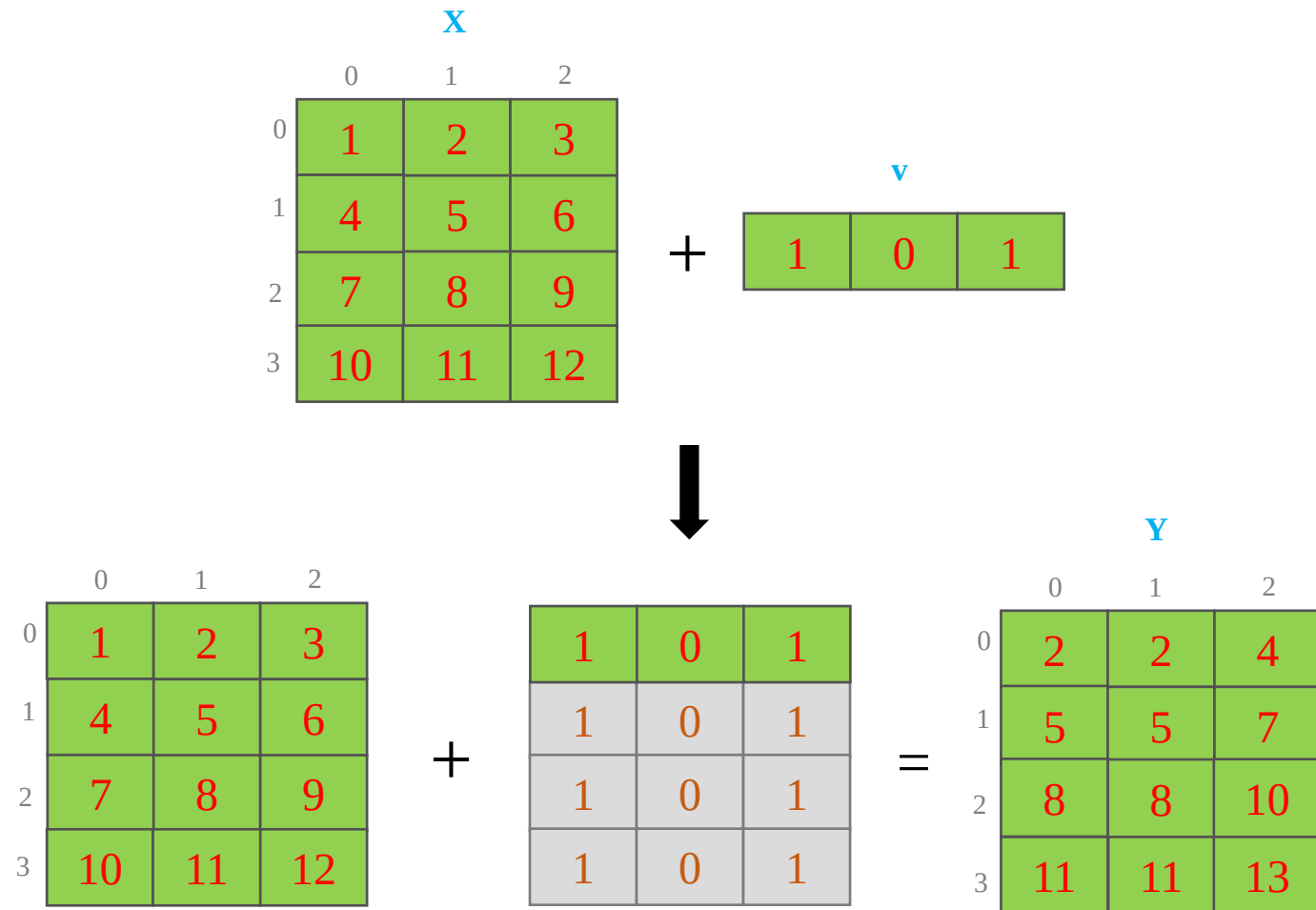
$$\begin{array}{c} \text{X} \\ \begin{array}{ccc} 0 & 1 & 2 \\ \hline 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \\ 10 & 11 & 12 \end{array} \end{array} + \begin{array}{c} \text{v} \\ \begin{array}{ccc} 1 & 0 & 1 \end{array} \end{array} = \begin{array}{c} \text{Y} \\ \begin{array}{ccc} 0 & 1 & 2 \\ \hline 2 & 2 & 4 \\ 5 & 5 & 7 \\ 8 & 8 & 10 \\ 11 & 11 & 13 \end{array} \end{array}$$

0
1
2
3

```
1 # aivietnam.ai
2 import numpy as np
3
4 X = np.array([[1, 2, 3],
5               [4, 5, 6],
6               [7, 8, 9],
7               [10, 11, 12]])
8 v = np.array([1, 0, 1])
9
10 Y = X + v
11 print(Y)
```

```
[[ 2  2  4]
 [ 5  5  7]
 [ 8  8 10]
 [11 11 13]]
```

❖ Matrix and vector



```

1 # aivietnam.ai
2 import numpy as np
3
4 X = np.array([[1, 2, 3],
5               [4, 5, 6],
6               [7, 8, 9],
7               [10, 11, 12]])
8 v = np.array([1, 0, 1])
9
10 Y = X + v
11 print(Y)

```

```

[[ 2  2  4]
 [ 5  5  7]
 [ 8  8 10]
 [11 11 13]]

```

Example 1: Softmax function

❖ Chuyển các giá trị của một vector thành các giá trị xác suất

(Stable) Formula			
$m = \max(x)$ $f(x_i) = \frac{e^{(x_i - m)}}{\sum_j e^{(x_j - m)}}$			
X	X-m		Probability
$x_1 = 1.0$	$x_1 = -2.0$	Softmax	$f(x_1) = 0.09$
$x_2 = 2.0$	$x_2 = -1.0$		$f(x_2) = 0.24$
$x_3 = 3.0$	$x_3 = 0$		$f(x_3) = 0.67$

```

1 import numpy as np
2
3 def stable_softmax(X):
4     exps = np.exp(X - np.max(X))
5     return exps / np.sum(exps)

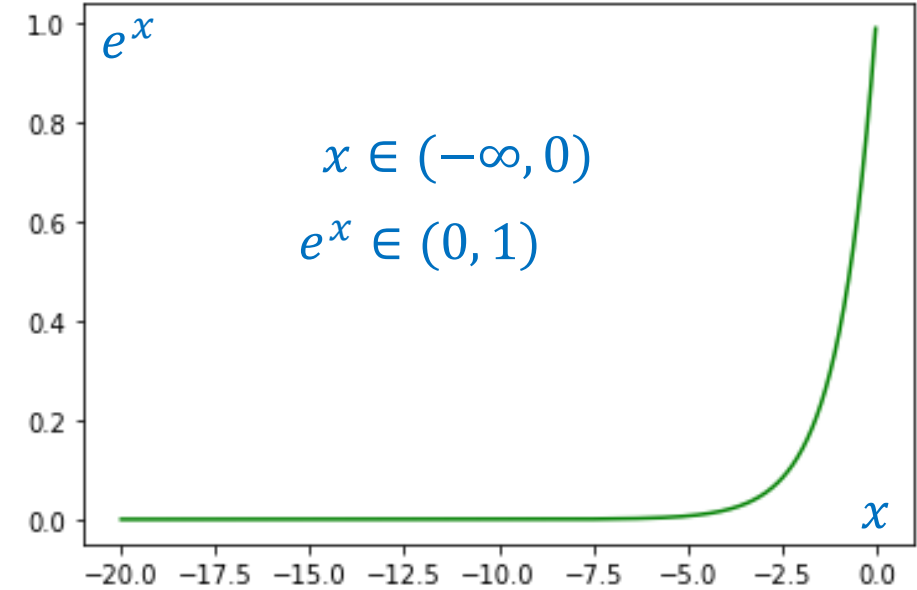
```

```

1 X = np.array([1.0, 2.0, 3.0])
2 f = stable_softmax(X)
3 print(f)

```

[0.09003057 0.24472847 0.66524096]



QUIZ TIME

Outline

SECTION 1

Introduction

SECTION 2

Common Functions

SECTION 3

Indexing and Broadcasting

SECTION 4

Examples

Feature	Label
Petal_Length	Category
1.4	0
1	0
1.5	0
3	1
3.8	1
4.1	1

One-hot encoding for label

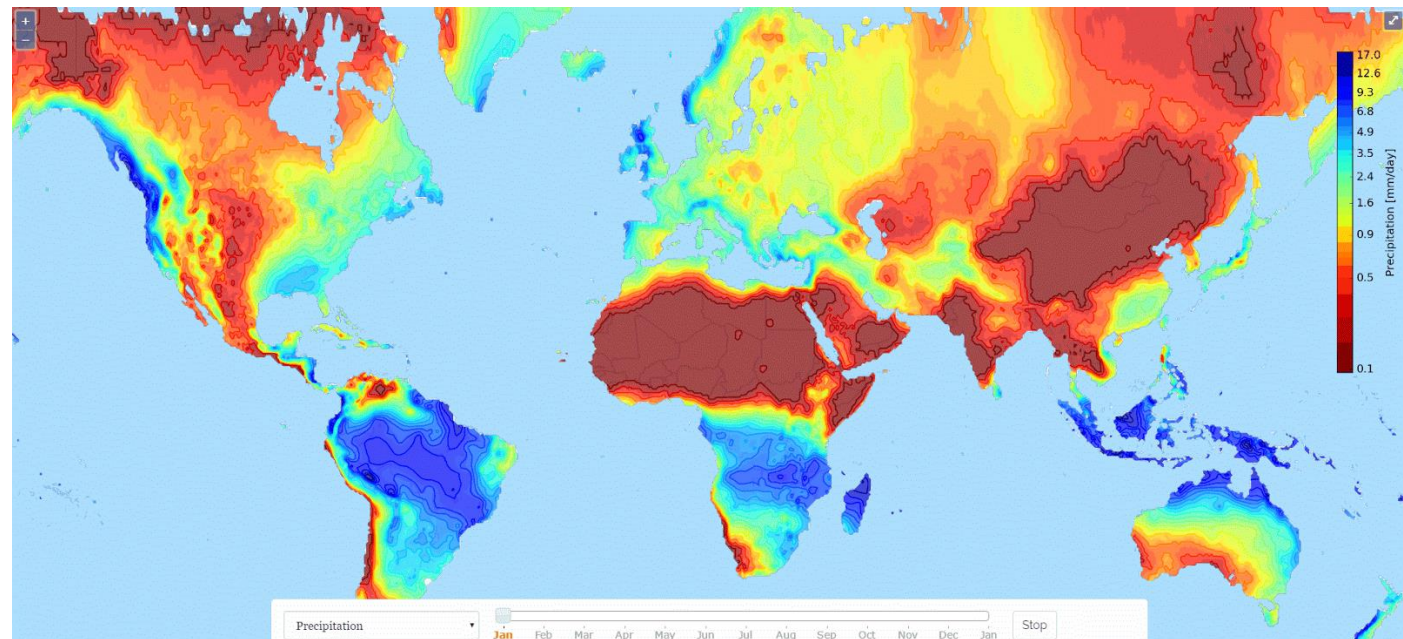
$$y = 0 \rightarrow \mathbf{y} = [1 \ 0]$$

$$y = 1 \rightarrow \mathbf{y} = [0 \ 1]$$

↑
scalar

↑
vector

❖ Weather forecasting



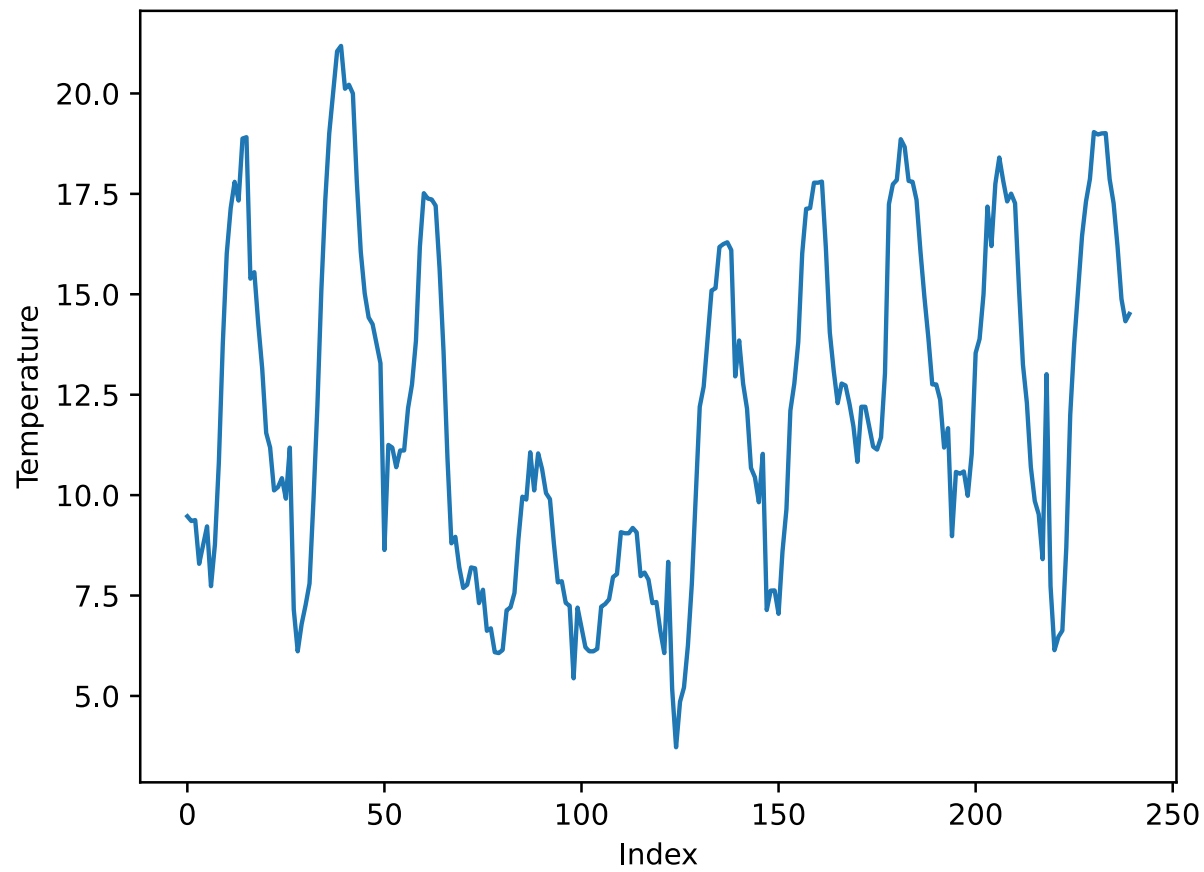
Predict future temperature in weather forecasting

❖ Weather forecasting

Date	Temperature (C)	No.	Temperature (C)
2006-04-01 00	9.472222222	0	9.472222222
2006-04-01 01	9.355555556	1	9.355555556
2006-04-01 02	9.377777778	2	9.377777778
2006-04-01 03	8.288888889	3	8.288888889
2006-04-01 04	8.755555556	4	8.755555556
2006-04-01 05	9.222222222	5	9.222222222
2006-04-01 06	7.733333333	6	7.733333333
2006-04-01 07	8.772222222	7	8.772222222
2006-04-01 08	10.82222222	8	10.82222222
2006-04-01 09	13.77222222	9	13.77222222
2006-04-01 10	16.01666667	10	16.01666667
2006-04-01 11	17.14444444	11	17.14444444
2006-04-01 12	17.8	12	17.8
2006-04-01 13	17.33333333	13	17.33333333
2006-04-01 14	18.87777778	14	18.87777778
2006-04-01 15	18.91111111	15	18.91111111
2006-04-01 16	15.38888889	16	15.38888889
2006-04-01 17	15.55	17	15.55
2006-04-01 18	14.25555556	18	14.25555556
2006-04-01 19	13.14444444	19	13.14444444
2006-04-01 20	11.55	20	11.55
2006-04-01 21	11.18333333	21	11.18333333
2006-04-01 22	10.11666667	22	10.11666667
2006-04-01 23	10.2	23	10.2

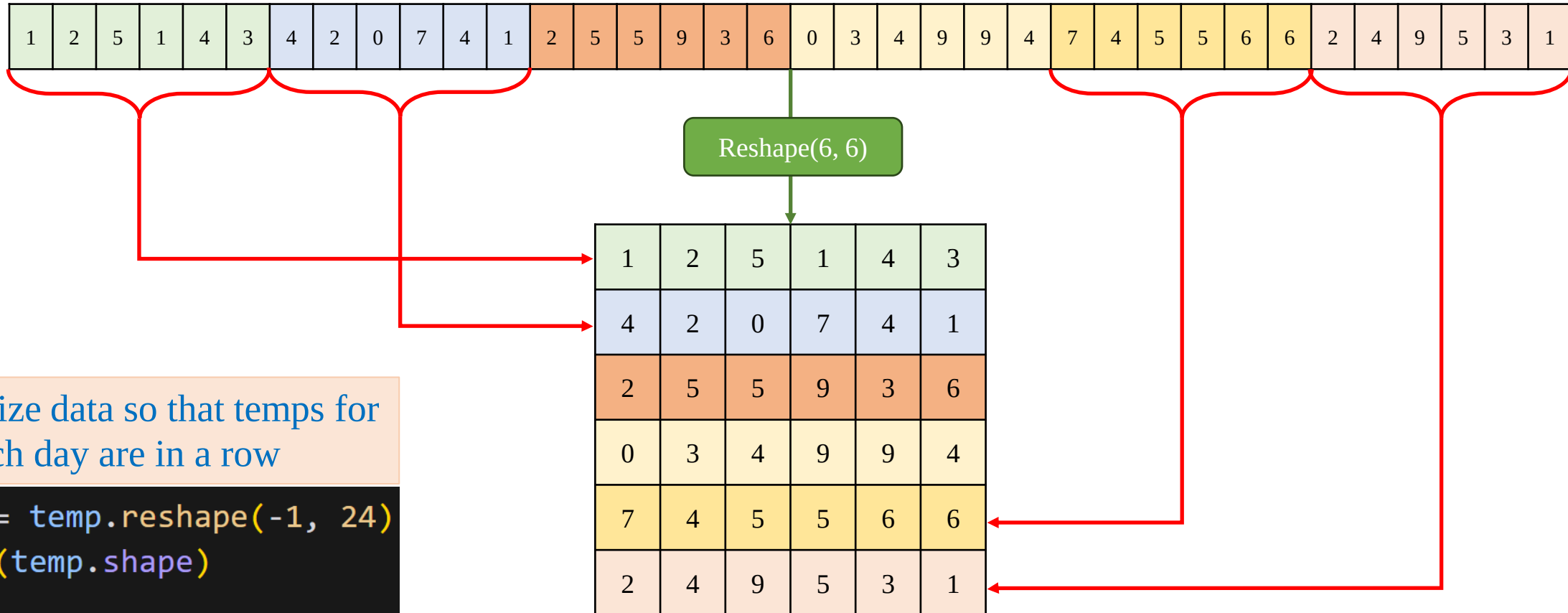
```
import pandas as pd
import numpy as np

data = pd.read_csv('temperature-1d.csv').to_numpy()
temp = data[:, 1]
```



Example 2

❖ Daily average temperatures



Re-organize data so that temps for each day are in a row

```
temp = temp.reshape(-1, 24)
print(temp.shape)
```

✓ 0.0s

(10, 24)

Reshape an array

❖ Daily average temperatures

```
import pandas as pd
import numpy as np

data = pd.read_csv('temperature-1d.csv').to_numpy()
temp = data[:, 1]
```

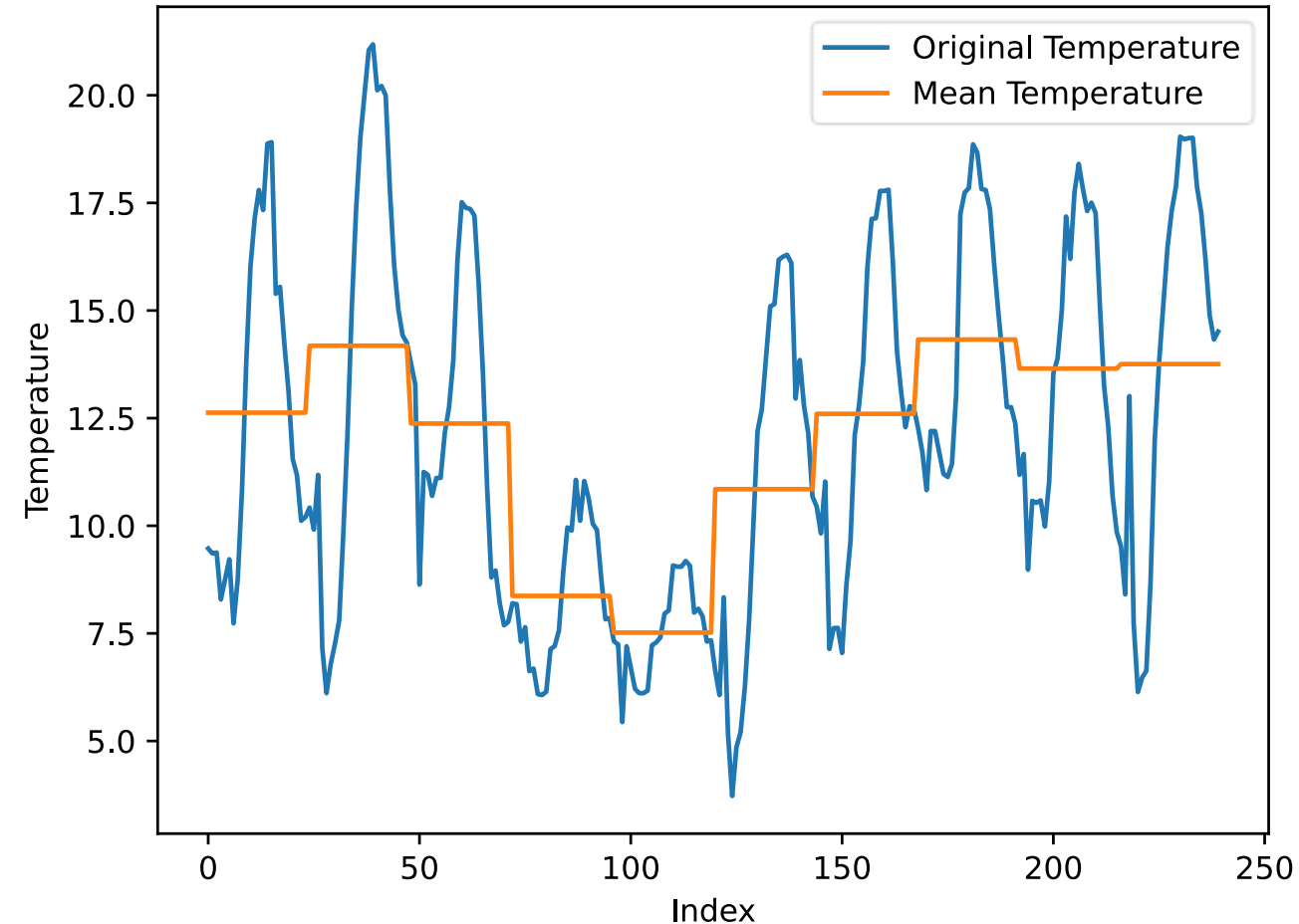
```
# Compute mean for each 24 data points
mean_24 = np.mean(temp.reshape(-1, 24), axis=1)
print(result.shape)
```

```
mean_24_repeat = np.repeat(mean_24, 24)
print(mean_24_repeat.shape)
```

✓ 0.0s

```
(240,)
(240,)
```

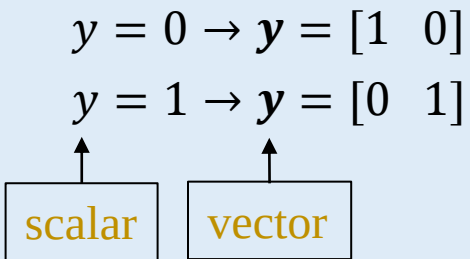
Temperature Data and Mean Temperature



❖ One-hot Encoding

Feature	Label	
Petal_Length	Category	
1.4	0	Yellow
1	0	
1.5	0	
3	1	Grey
3.8	1	
4.1	1	

One-hot encoding for label



Feature		Label	
Petal_Length	Petal_Width	Label	
1.5	0.2	1	Yellow
1.4	0.2	1	
1.6	0.2	1	
4.7	1.6	2	Grey
3.3	1.1	2	
4.6	1.3	2	
5.6	2.2	3	Purple
5.1	1.5	3	
5.6	1.4	3	

One-hot encoding for label

$$y = 0 \rightarrow \mathbf{y} = [1 \ 0 \ 0]$$

$$y = 1 \rightarrow \mathbf{y} = [0 \ 1 \ 0]$$

$$y = 2 \rightarrow \mathbf{y} = [0 \ 0 \ 1]$$

One-hot Encoding

Feature		Label
Petal_Length	Petal_Width	Label
1.4	0.2	0
1.4	0.2	0
1.3	0.2	0
4.5	1.5	1
4.9	1.5	1
4	1.3	1
4.5	1.7	2
6.3	1.8	2
5.8	1.8	2

#classes=3

#samples=9

One-hot encoding for label

$y = 0 \rightarrow \mathbf{y} = [1 \ 0 \ 0]$

$y = 1 \rightarrow \mathbf{y} = [0 \ 1 \ 0]$

$y = 2 \rightarrow \mathbf{y} = [0 \ 0 \ 1]$

```
1 import numpy as np
2
3 #num_labels
4 K = 3
5
6 # num_samples
7 N = 9
8
9 labels = [0, 0, 0, 1, 1, 1, 2, 2, 2]
10 labels = np.array(labels)
11 print(labels)
```

[0 0 0 1 1 1 2 2 2]

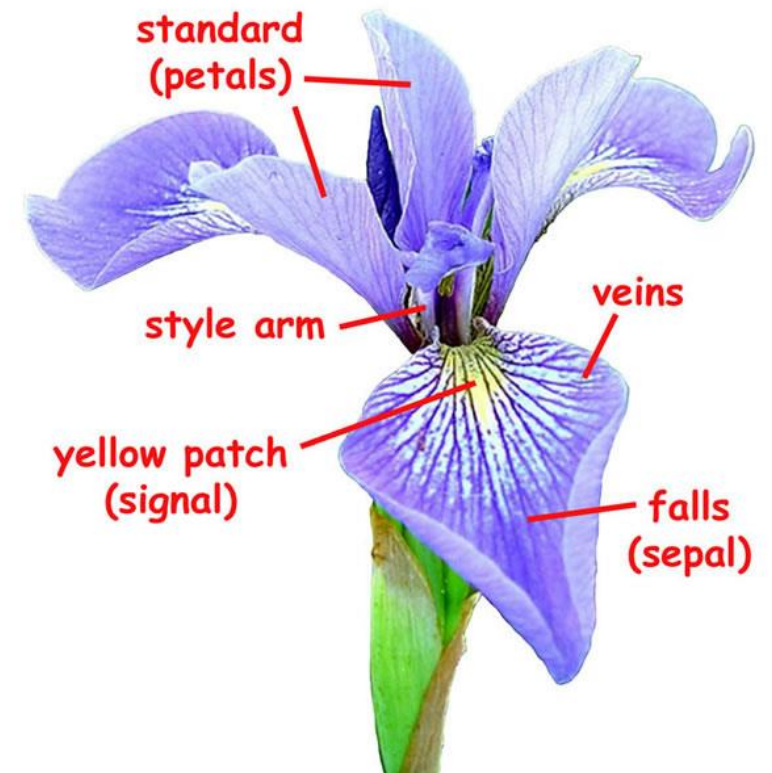
```
1 # create an NxK array
2 labels_onehot = np.zeros((N, K))
3
4 # one-hot encoding
5 labels_onehot[np.arange(N), labels] = 1
6 print(labels_onehot)
```

```
[[1.  0.  0.]
 [1.  0.  0.]
 [1.  0.  0.]
 [0.  1.  0.]
 [0.  1.  0.]
 [0.  1.  0.]
 [0.  0.  1.]
 [0.  0.  1.]
 [0.  0.  1.]]
```

❖ Text data

❖ IRIS data

sepal_length	sepal_width	petal_length	petal_width	species
5.1	3.5	1.4	0.2	Iris-setosa
4.9	3	1.4	0.2	Iris-setosa
4.7	3.2	1.3	0.2	Iris-setosa
4.6	3.1	1.5	0.2	Iris-setosa
5	3.6	1.4	0.2	Iris-setosa
5.4	3.9	1.7	0.4	Iris-setosa
4.6	3.4	1.4	0.3	Iris-setosa
5	3.4	1.5	0.2	Iris-setosa
4.4	2.9	1.4	0.2	Iris-setosa
4.9	3.1	1.5	0.1	Iris-setosa



❖ Some functions

sepal_length	sepal_width	petal_length	petal_width	species
5.1	3.5	1.4	0.2	Iris-setosa
4.9	3	1.4	0.2	Iris-setosa
4.7	3.2	1.3	0.2	Iris-setosa
4.6	3.1	1.5	0.2	Iris-setosa
5	3.6	1.4	0.2	Iris-setosa
5.4	3.9	1.7	0.4	Iris-setosa
4.6	3.4	1.4	0.3	Iris-setosa
5	3.4	1.5	0.2	Iris-setosa
4.4	2.9	1.4	0.2	Iris-setosa
4.9	3.1	1.5	0.1	Iris-setosa

```
np.genfromtxt(fname='...',  
              dtype=float, delimiter=',',  
              skip_header=1,  
              usecols=[0, 1, 2, 3])
```

```
np.genfromtxt(fname='...',  
              dtype=float, delimiter=',',  
              skip_header=1,  
              usecols=4)
```

❖ Some functions

Find the unique elements

`np.unique(data)`

1	2	3	2	1
---	---	---	---	---

data

`np.unique(data) =`

1	2	3
---	---	---

```
data = np.array([1, 2, 3, 2, 1])
print(data)
```

```
result = np.unique(data)
print(result)
```

✓ 0.0s

```
[1 2 3 2 1]
[1 2 3]
```

'class-1'	'class-2'	'class-3'	'class-2'
-----------	-----------	-----------	-----------

data

result =

'class-1'	'class-2'	'class-3'
-----------	-----------	-----------

```
data = np.array(['class-1', 'class-2',
                 'class-3', 'class-2'])
print(data)
```

```
result = np.unique(data)
print(result)
```

✓ 0.0s

```
['class-1' 'class-2' 'class-3' 'class-2']
['class-1' 'class-2' 'class-3']
```


❖ Some functions

Cast to a specified type

`np.ndarray.astype(dtype=float)`

For each element in **data**,
replace **old** by **new**

`np.char.replace(data,
old,
new)`

data =

'class-1'	'class-2'	'class-3'
-----------	-----------	-----------

data =

'0'	'1'	'2'
-----	-----	-----

data =

0	1	2
---	---	---

```
data = np.array(['class-1', 'class-2', 'class-3'])
data = np.char.replace(data, 'class-1', '0')
data = np.char.replace(data, 'class-2', '1')
data = np.char.replace(data, 'class-3', '2')
print(data)
```

```
data = data.astype(float)
print(data)
```

✓ 0.0s

```
['0' '1' '2']
```

```
[0. 1. 2.]
```


IRIS data

col 4

↕ species

[illegible]

